

Концепт: Decision Tree & Random Forest — Регрессия стоимости жилья

Этот файл объясняет каждый блок ноутбука `Regression-for-task4_part_2.ipynb` — что делается, зачем и как это работает под капотом.

Оглавление

1. [Импорты](#)
2. [Загрузка данных и EDA](#)
3. [Предобработка данных](#)
4. [Разделение данных](#)
5. [Decision Tree — подбор параметров \(GridSearchCV\)](#)
6. [Decision Tree — оценка модели](#)
7. [Random Forest — подбор числа деревьев](#)
8. [Random Forest — оценка модели](#)
9. [Важность признаков](#)
10. [Сравнение моделей](#)
11. [Предсказание нового дома](#)

1. Импорты

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Библиотека	Зачем
numpy	Вычисление RMSE через <code>np.sqrt</code> , сортировка <code>np.argsort</code>
matplotlib / seaborn	Графики (scatter, histogram, bar chart)
pandas	Датафреймы, чтение CSV
train_test_split	Разбиение на обучение и тест
GridSearchCV	Автоматический перебор параметров с кросс-валидацией
DecisionTreeRegressor	Дерево решений для регрессии
plot_tree	Визуализация структуры дерева
RandomForestRegressor	Случайный лес для регрессии
mean_absolute_error	MAE
mean_squared_error	MSE
r2_score	R ² (коэффициент детерминации)

Почему нет StandardScaler ?

Деревья решений и случайный лес принимают решения по пороговым значениям признаков (`median_income > 3.5`), а не по расстоянию до соседей.

Масштаб признаков на результат не влияет — скейлинг не нужен.

2. Загрузка данных и EDA

Датасет: housing.csv

```
df = pd.read_csv('housing.csv')
```

- **20 640 домов** Калифорнии.
- 9 числовых признаков + 1 категориальный (`ocean_proximity`).
- Цель: `median_house_value` (медианная стоимость домов в блоке, \$).

Первичный осмотр

```
df.info()          # типы, Non-Null Count
df.isnull().sum()  # пропуски
df.describe()      # мин/макс/среднее
```

Что обнаруживаем:

- `total_bedrooms` — 207 пропусков ($\approx 1\%$). Заполняем медианой.
- `ocean_proximity` — строковый тип, нужно закодировать.
- `median_house_value` — распределение со "хвостом" справа и пиком у 500 000 (это значение используется как потолок в данных).

Графики EDA

```
sns.histplot(df['median_house_value'], ...) # распределение цены
sns.heatmap(df.corr(), ...)                # корреляции
sns.scatterplot(x='median_income', y='median_house_value', ...) # зависимость
sns.boxplot(x='ocean_proximity', y='median_house_value', ...)  # цена по зонам
```

Что видим:

- `median_income` — самый коррелирующий с ценой признак (~ 0.69).
- Дома у океана стоят дороже чем inland.
- Большой разброс цен в одном блоке.

3. Предобработка данных

Заполнение пропусков

```
df['total_bedrooms'] = df['total_bedrooms'].fillna(df['total_bedrooms'].median())
```

Медиана устойчива к выбросам, поэтому это предпочтительнее среднего.

One-Hot Encoding

```
df = pd.get_dummies(df, columns=['ocean_proximity'], drop_first=True)
```

Создаёт бинарные колонки:

Исходное значение	Новые колонки (True/False)
<1H OCEAN	(базовый, удалён через drop_first)
INLAND	ocean_proximity_INLAND
ISLAND	ocean_proximity_ISLAND
NEAR BAY	ocean_proximity_NEAR BAY
NEAR OCEAN	ocean_proximity_NEAR OCEAN

drop_first=True убирает одну категорию чтобы избежать мультиколлинеарности.
Если все 4 дополнительных признака = 0, то это <1H OCEAN .

Feature Engineering

```
df['rooms_per_household'] = df['total_rooms'] / df['households']
df['bedrooms_per_room'] = df['total_bedrooms'] / df['total_rooms']
df['population_per_household'] = df['population'] / df['households']
```

Зачем: Сырые счётчики (total_rooms = 1500) ничего не говорят без контекста.
А вот rooms_per_household = 5.5 — это уже характеристика отдельного дома.

Признак	Формула	Смысл
rooms_per_household	total_rooms / households	Средний размер жилья
bedrooms_per_room	total_bedrooms / total_rooms	Доля спален (тип жилья)
population_per_household	population / households	Плотность заселения

После этого исходные счётчики удаляются.

Удаление выбросов

```
df = df[df['median_house_value'] < 500_000]
```

Значение 500 000 — это "заглушка" (capped). Все дома дороже просто ставились на 500k. Оставлять их в обучении нежелательно — модель будет "знать потолок" и предсказывать его.

4. Разделение данных

```
X = df.drop('median_house_value', axis=1)
y = df['median_house_value']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- 80% данных → обучение, 20% → тест.
- random_state=42 — фиксирует разбиение для воспроизводимости.

5. Decision Tree — подбор параметров

Что такое Decision Tree Regressor

Дерево решений для регрессии работает так же как для классификации — делит данные цепочкой условий. Но вместо класса на листе — **среднее значение** целевой переменной всех объектов, попавших в этот лист.

```
median_income <= 3.5?
├─ Да → rooms_per_household <= 4.0?
│   ├── Да → предсказание: 120 000
│   └─ Нет → предсказание: 155 000
└─ Нет → latitude <= 34.5?
    ├── Да → предсказание: 280 000
    └─ Нет → предсказание: 380 000
```

GridSearchCV — три параметра

```
param_grid_dt = {  
    'max_depth':      [3, 5, 7, 10, 15, None],  
    'min_samples_leaf': [1, 5, 10, 20],  
    'max_features':    [None, 'sqrt', 'log2']  
}
```

Итого: $6 \times 4 \times 3 = 72$ комбинации $\times$ 5 фолдов = 360 обучений.

max_depth — максимальная глубина дерева.

- None — дерево растёт пока не разделит все объекты. Риск переобучения.
- 3–5 — неглубокое дерево, недообучение.
- 7–10 — обычно оптимальная зона.

min_samples_leaf — минимальное число объектов в листе.

- 1 — дерево может создать лист из одного объекта → переобучение.
- 10–20 — каждый лист должен иметь хотя бы 10–20 объектов → сглаживает предсказания.

max_features — сколько признаков рассматривать в каждом узле.

- None — все признаки → "жадное" дерево.
- 'sqrt' / 'log2' — только подмножество → меньше переобучение.

Метрика оптимизации

```
scoring='neg_mean_squared_error'
```

GridSearchCV **максимизирует** метрику. Поскольку MSE нужно минимизировать, передаём отрицательное значение: лучший результат = наибольшее `neg_MSE` = наименьший MSE.

```
RMSE_CV = np.sqrt(-grid_dt.best_score_)
```

6. Decision Tree — оценка модели

Метрики регрессии

```
mae = mean_absolute_error(y_test, dt_pred) # средняя абс. ошибка
mse = mean_squared_error(y_test, dt_pred)  # среднеквадр. ошибка
rmse = np.sqrt(mse)                        # корень из MSE
r2 = r2_score(y_test, dt_pred)             # коэф. детерминации
```

Метрика	Формула	Смысл
MAE	$\text{mean}(y - \hat{y})$	$y - \hat{y}$
MSE	$\text{mean}((y - \hat{y})^2)$	Штрафует большие ошибки сильнее. Труднее интерпретировать.
RMSE	$\sqrt{\text{MSE}}$	Та же шкала что MAE, но чувствительнее к выбросам.
R²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	1.0 = идеально, 0 = модель = константа (среднее).

Хорошие ориентиры для этого датасета:

- $\text{RMSE} < 50\,000 \rightarrow$ хорошо
- $R^2 > 0.8 \rightarrow$ модель объясняет >80% дисперсии

График остатков

```
residuals = y_test - dt_pred
plt.scatter(dt_pred, residuals)
plt.axhline(0, color='red', linestyle='--')
```

Что смотреть:

- Идеальный случай: точки случайно разбросаны вокруг 0, без паттернов.
- "Воронка" (fan shape) — ошибка растёт с ценой → гетероскедастичность.
- Систематический сдвиг → модель регулярно завышает/занижает.

Гистограмма остатков должна быть симметричной с центром в 0.

7. Random Forest — подбор числа деревьев

Что такое Random Forest Regressor

Random Forest для регрессии — это ансамбль деревьев:

1. **Bootstrap**: из N обучающих объектов случайно берём N объектов с возвращением. Примерно 63% уникальных.
2. **Random subspace**: в каждом узле рассматриваем только $\sqrt{p}$ случайных признаков.
3. Строим дерево.
4. Повторяем $n_estimators$ раз.
5. **Итоговое предсказание** = **среднее** по всем деревьям.

Почему это лучше одного дерева:

- Каждое дерево обучается на немного другой выборке → разные ошибки.
- Усредняя, мы "сглаживаем" ошибки — снижаем дисперсию (variance).
- Одно дерево легко переобучается; лес — устойчивее.

GridSearchCV для $n_estimators$

```
param_grid_rf = {'n_estimators': [10, 50, 100, 200, 300]}
```

- При 10 деревьях — лес ещё "шумит", предсказания нестабильны.
- При 100–200 деревьях — качество обычно выходит на плато.
- 300+ деревьев — маргинальный прирост, дольше обучение.

8. Random Forest — оценка модели

Те же четыре метрики: MAE, MSE, RMSE, R^2 .

RF обычно значительно лучше одного дерева:

- Меньше MAE и RMSE → точнее.
- Выше R^2 → объясняет больше дисперсии.

Графики остатков у RF как правило равномернее распределены — меньше выбросов и паттернов.

9. Важность признаков

```
importances = rf.feature_importances_
```

После обучения RF каждому признаку назначается важность — насколько сильно он в среднем снижал MSE при разбиениях во всех деревьях. Сумма = 1.0.

```
indices = np.argsort(importances)[::-1] # убывающий порядок
plt.bar(range(len(importances)), importances[indices])
plt.xticks(range(len(importances)), feature_names[indices], rotation=45)
```

Ожидаемый результат: median_income должен быть самым важным признаком — он имеет самую высокую корреляцию с целевой переменной (~0.69).

Почему RF, а не DT для важности?

RF усредняет важности по сотням деревьев → оценка стабильнее, чем у одного дерева.

10. Сравнение моделей

```
comparison = pd.DataFrame({
    'Модель': ['Decision Tree', 'Random Forest'],
    'MAE':     [dt_mae, rf_mae],
    'RMSE':    [dt_rmse, rf_rmse],
    'R²':      [dt_r2, rf_r2]
})
comparison.style.highlight_min(axis=0, color='lightgreen')
```

highlight_min — зелёным подсвечивает наименьшую ошибку в каждом столбце.
Для R² следует смотреть наибольшее значение (это минус таблицы — highlight_max нужен отдельно).

Столбчатый график наглядно показывает насколько Random Forest лучше одного дерева.

11. Предсказание нового дома

```
new_house = {  
    'longitude': -118.5,  
    'latitude': 34.0,  
    'housing_median_age': 20.0,  
    'median_income': 5.0,  
    ...  
}
```

```
new_house_df = pd.DataFrame([new_house], columns=X.columns)
```

```
dt_price = dt.predict(new_house_df)[0]
```

```
rf_price = rf.predict(new_house_df)[0]
```

- Оборачиваем в DataFrame с правильными именами колонок — sklearn ожидает именно такой формат.
- `predict()` для регрессора возвращает числовое значение, а не класс.
- Сравниваем предсказания двух моделей на одном примере.

Итоговая схема пайплайна

Данные (housing.csv, ~20 000 домов, 10 признаков)



EDA (info, describe, histplot, heatmap, scatterplot, boxplot)



Предобработка:

- fillna(median) для total_bedrooms
- get_dummies для ocean_proximity
- Feature Engineering (rooms/bedrooms/population per household)
- Фильтрация выбросов (< 500 000)
- Удаление сырых счётчиков



train_test_split (80% / 20%, NO StandardScaler)



Decision Tree Regressor
GridSearchCV(max_depth,
min_samples_leaf,
max_features)
→ plot_tree (визуализация max_depth=3)
→ MAE, MSE, RMSE, R ²
→ Actual vs Predicted plot
→ Residuals plot + histogram



Random Forest Regressor
GridSearchCV(n_estimators)
→ MAE, MSE, RMSE, R ²
→ Actual vs Predicted plot
→ Residuals plot + histogram
→ Feature Importance



Сравнение: DT vs RF (таблица + bar chart)



Предсказание нового дома

Ключевые различия: Регрессия vs Классификация

Аспект	Классификация (Part 1)	Регрессия (Part 2)
Целевая переменная	Категория (0, 1, 2)	Число (\$)
Метрика оптимизации GridSearchCV	accuracy	neg_mean_squared_error
Предсказание листа	Мажоритарный класс	Среднее значение
Метрики оценки	Accuracy, Precision, Recall, F1, AUC-ROC	MAE, MSE, RMSE, R ²
Анализ ошибок	Confusion matrix	График остатков (residuals)
Пороги	Оптимальный порог по Юдену	Не применимо